

```

{*****}
{*
{*          -----0
{*          -----000
{*          -----0
{*
{*
{*
{*  -----TTTTT---EEEE---TTTT---RRRR---III---SSS
{*      TTT   EEE   TTT   RRR R   III   SSSSS
{*  ---TTT---EEEE---TTT---RRRR---III---SS S
{*      TTT   EEE   TTT   RRR R   III   SS
{*      TTT   EEEEE  TTT   RRR R   III   SS
{*      TTT   EEEEE  TTT   RRR R   III   S  SS
{*  -----SSSSS
{*  -----SSS
{*
{*
{*      Projekt PC Kurs
{*      Lehrwerkstatt Alcatel STR AG
{*      23. November bis 18. Dezember MXMII
{*      (C) Claudio Zopfi 6.989
{*
{*****}

```

```

program Tetris;
uses graph,dos,crt;

```

```

type  tTopFive = record
        Name  :String[20];
        Score :LongInt;
    end;
    tRotVect = record
        vX,
        vY,
        vix,
        viy,
        vjx,
        vjy  :shortInt;
    end;
    tAngel    = (up,right,down,left);

```

```

const path          :string[20]    =  '';
    XKloz           = 10;           {Filename f•r Top5}
                                   {Anzahl Spielfelder auf X-Achse}

```

```

YKloz                = 25;                {Anzahl Spielfelder auf Y-Achse}
AnzStones             = 7;                {Anzahl Verschiedene Spielsteine}
StoneBreite          = 2;                {Breite Spielstein}
StoneHoehe           = 4;                {H"he Spielstein}
Tackt                = 150;              {Verz"gerungszeit [ms]}
XStp                  :byte              = 10;    {Pixels pro Spielfeld (X-Richtung)}
YStp                  :byte              = 7;      {Pixels pro Spielfeld (Y-Richtung)}
XChr                  = 50;              {Punkte-Position}
YStpChr              :integer            = 50;    {Abstand Zeilen}
XField                = 250;             {X-Position SpielRahmen [Pixels]}
YField                = 50;              {Y-Position SpielRahmen [Pixels]}
FieldBreite           = 128;             {Breite SpielRahmen [Pixels]}
FieldHoehe            = 250;             {H"he SpielRahmen [Pixels]}
FieldRand             = 20;              {Randbreite SpielRahmen [Pixels]}
Level                 :byte              = 1;      {SpielStufe}
AutoInk               :boolean           = true;   {Automatisch schnell Schalter}
ShowNext              :boolean           = true;   {N"chster Stein Anzeigen}
GrideColor            :byte              = Black;  {Gitterfarbe}
BGColor               :byte              = White;  {Hintergrundfarbe}
StatusColor           :byte              = Red;    {PunktZahlFarbe}
ChrColor              :byte              = Black;  {SchriftenFarbe}
StoneBitMap           :string= ' * . . . . . . . '+'
                        ' * . * ** * . * '+'
                        ' * . * ** ** ** ** '+'
                        ' * . ** ** . * . * '+';

var TopFive           :File              of tTopFive;
    TopFiveMem         :array[1..5]      of tTopFive;
    PlayField          :array[1..30,1..40] of byte;    {Boolsches Spielfeld}
    Stones              :array[1..AnzStones,1..StoneBreite,
                                1..StoneHoehe] of boolean; {Speicher f"r SteineLookALike}

    LastX,LastY,
    LastStyle          :byte;            {Vorg"nger}
    LastAngl           :tAngel;
    RotVectors         :tRotVect;        {globale Rotationsvektoren}

{*****}
{ *      Initialisiert Grafik Modus      * }
{*****}
{ * }
{ * }
{ *      (C) 27.11.92      * }

```

```

{*****}
procedure Init;
var GraphDriver, GraphMode    :integer;
    Err                        :integer;
begin
    GraphDriver:=Detect;
    InitGraph(GraphDriver, GraphMode, '');
    Err:=GraphResult;
    if Err<>grOk then
    begin
        Writeln('Fehler in Grafik : ', GraphErrorMsg(Err));
        Halt(1);
    end;
end;

{*****}
{*      Spielsteine initialisieren      *}
{*****}
{*      PM: In: Bitmap in Form von String      *}
{*      Steinbreite/h"he      *}
{*      AnzahlSteine      *}
{*      Out:Array of Bit      *}
{*      (C) 30.11.92      *}
{*****}
procedure InitStones(StoneBitMap :string);
var  x,y,n      :integer;
begin
    For y:=0 to StoneHoehe-1 do
    For n:=0 to AnzStones-1 do
    For x:=0 to StoneBreite-1 do
        if StoneBitMap[4*(n+y*AnzStones)+x+1]='*' then Stones[n+1,x+1,y+1]:=true
        else Stones[n+1,x+1,y+1]:=false;
end;

procedure testInitStones;
var i,j,k :integer;
begin
    For i:=1 to AnzStones do
    begin
        clrscr;
        for j:=1 to 2 do

```

```

        for k:=1 to 4 do
        if Stones[i,j,k] then
        begin
            gotoxy(j,k);
            write('*');
        end;
        readln;
    end;
end;

procedure testInitStones2;
var i,j,k :integer;
begin
    SetColor(Red);
    For i:=1 to AnzStones do
    begin
        clearviewport;
        for j:=1 to 2 do
        for k:=1 to 4 do
        if Stones[i,j,k] then
        begin
            bar(j*XStp,k*YStp,(j+1)*XStp,(k+1)*YStp);
        end;
        readln;
    end;
end;
end;

```

```

{*****}
{*      Schreiben auf Bildschirm      *}
{*****}
{*      PM: In: Strint                *}
{*              X/Y-Position          *}
{*              Richtung (Horiz/Vert) *}
{*              Schrifth"he           *}
{*              Farbe                 *}
{*                                  *}
{*      (C) 27.11.92                  *}
{*****}
procedure Print(Txt :string; X,Y :integer; Font,Direction,Hi,color :word);
begin
    SetColor(Color);

```



```

    SetTextStyle(Font,Direction,Hi);
    OutTextXY(X,Y,Txt);
end;

{*****}
{*      TopFive von Disk laden und darstellen.      *}
{*****}
{*      (C) 30.11.92      *}
{*****}
procedure InitTopFive;
var i      :integer;
    OneOfFive :tTopFive;
begin
    Assign(TopFive,path+'Top5.dat');
    rewrite(TopFive);
    OneOfFive.Score:=6000;
    for i:=1 to 5 do
        with OneOfFive do
            begin
                Name:='Claudio Zopfi    !!!';
                Score:=Score-1000;
                write(TopFive,OneOfFive);
            end;
        close(TopFive);
    end;

procedure LoadTopFive;
var Y      :Integer;
    sScr   :string[20];
begin
    path:=FSearch('top5.dat',GetEnv('Path'));
    if path='' then
        begin
            InitTopFive;
            path:=FSearch('top5.dat',GetEnv('Path'));
        end;
    Assign(TopFive,path);
    reset(TopFive);
    for Y:=1 to 5 do
        begin
            read(TopFive,TopFiveMem[Y]);

```

```

    end;
    close(TopFive);
end;

procedure ShowTopFive;
var y      :integer;
    sScr :String[20];
begin
    print('TopFive',100,150,TriplexFont,HorizDir,3,ChrColor);
    LoadTopFive;
    for Y:=1 to 5 do
        with TopFiveMem[y] do
            begin
                Str(Score,sScr);
                print(Name,50,160+Y*20,TriplexFont,HorizDir,1,6-Y);
                print(sScr,230,160+Y*20,TriplexFont,HorizDir,1,6-Y);
            end;
        end;
    end;
end;

{*****}
{*      Top Five aktualisieren      *}
{*****}
{*      PM: In: Punktestand          *}
{*      *}                           *}
{*      (C) 10.12.92                 *}
{*****}
procedure AktuTopFive(NewScore :LongInt);
var i,j :integer;
    Nam :string[15];
begin
    LoadTopFive;
    i:=0;
    repeat
        inc(i);
        if NewScore>TopFiveMem[i].Score then
            begin
                SetColor(ChrColor);
                for j:=4 downto i do TopFiveMem[j+1]:=TopFiveMem[j];
                SetFillStyle(SolidFill,StatusColor);
                bar(XField,YField,XField+XStp*XKloz,YField+YStp*YKloz);
                Print('Herzliche',XField,YField,TriplexFont,HorizDir,1,ChrColor);
                Print('Gratulation !!!!!',XField,YField+YStpChr Div 2,TriplexFont,HorizDir,1,ChrColor);
            end;
        end;
    repeat

```

```

Print('Du kannst',XField,YField+YStpChr,TriplexFont,HorizDir,1,ChrColor);
Print('Deinen Namen',XField,YField+YStpChr*3 div 2,TriplexFont,HorizDir,1,ChrColor);
Print('eingeben:',XField,YField+YStpChr*2,TriplexFont,HorizDir,1,ChrColor);
gotoxy(32,12);
readln(TopFiveMem[i].Name);
TopFiveMem[i].Score:=NewScore;
exec('del',path);
Assign(TopFive,path);
rewrite(TopFive);
for i:=1 to 5 do write(TopFive,TopFiveMem[i]);
close(TopFive);
end;
until (NewScore>TopFiveMem[i].Score) or (i=5);
end;

{*****}
{* Spielfeld darstellen *}
{*****}
{* PM: In: H"he/Breite *}
{* Gitterschalter *}
{* Out:H"he/Breite eines Spielfeldes *}
{* *}
{* (C) 30.11.92 *}
{*****}
procedure ShowPlayField;
const X = XField; {250;}
      Y = YField; {50;}
      Xmax = FieldBreite; {300;}
      ymax = FieldHoehe; {250;}
      Rand = FieldRand; {20;}
var Hoehe,
     Breite,i,j :byte;
begin
  ClearViewPort;
  Hoehe:=YKloz;
  Breite:=XKloz;
  XStp:=Xmax div Breite;
  YStp:=Ymax div Hoehe;

  { **** Hintergrund **** }

  SetFillStyle(SolidFill,BGColor);

```

```

Bar(0,0,GetMaxX,GetMaxY);

{ **** Rahmen zeichnen **** }

SetColor(Red);
SetFillStyle(SolidFill,Red);
bar(x-Rand,y,x,y+Rand+Hoehe*YStp);
bar(x-Rand,y+Hoehe*YStp,x+Rand+Breite*XStp,y+Rand+Hoehe*YStp);
bar(x+Breite*XStp,y,x+Rand+breite*XStp,y+Rand+Hoehe*YStp);

{ **** Gitter zeichnen **** }

SetColor(GrideColor);
SetLineStyle(SolidLn,0,0);
for i:=0 to Hoehe do line(x,y+i*YStp,x+Breite*XStp,y+i*YStp);
for i:=0 to Breite do line(x+i*XStp,y,x+i*XStp,y+Hoehe*YStp);

{ **** Beschriftung **** }

YStpChr:=ymax div 6;
print('Score:',XChr,Y,TriplexFont,HorizDir,1,ChrColor);
print('000000',XChr,Y+YStpChr,SansSerifFont,HorizDir,1,StatusColor);
print('Stones:',XChr,Y+2*YStpChr,TriplexFont,HorizDir,1,ChrColor);
print('000000',XChr,Y+3*YStpChr,SansSerifFont,HorizDir,1,StatusColor);
print('Lines:',XChr,Y+4*YStpChr,TriplexFont,HorizDir,1,ChrColor);
print('000000',XChr,Y+5*YStpChr,SansSerifFont,HorizDir,1,StatusColor);
print('Tackt:',XField+(XKloz+5)*XStp,YField+2*YStpChr,TriplexFont,HorizDir,1,ChrColor);
print('150 ms',XField+(XKloz+5)*XStp,YField+3*YStpChr,TriplexFont,HorizDir,1,StatusColor);
print('Level:',XField+(XKloz+5)*XStp,YField+4*YStpChr,TriplexFont,HorizDir,1,ChrColor);
print('1',XField+(XKloz+5)*XStp,YField+5*YStpChr,TriplexFont,HorizDir,1,StatusColor);
print('*   T   E   T   R   I   S   ',X-Rand-5,Y,GothicFont,VertDir,3,ChrColor);
print('*   T   E   T   R   I   S   ',X+Breite*XStp-5,Y,GothicFont,VertDir,3,ChrColor);
end;

{ **** }
{ *   Rotations Vektoren berechnen   * }
{ **** }
{ *   PM: In: Winkel (90 Grad Abstufung)   * }
{ *   SteinH"he/Breite   * }
{ *   Out:Verschiebung der Punkte X und Y   * }
{ *   Einfluss von i und j auf X-Achse   * }
{ *   Einfluss von i und j auf Y-Achse   * }

```

```

{ *
{ *      (C) 3.12.92
{ *****
procedure SetRotationVectors (Angel :tAngel);
begin
    with RotVectors do
    Case Angel of
        Up:
            begin
                vX:=0;    vY:=0;
                vix:=1;   vjx:=0;
                viy:=0;   vjy:=1;
            end;
        Right:
            begin
                vX:=StoneBreite div 2+StoneHoehe div 2-1;
                vY:=StoneHoehe div 2-StoneBreite div 2;
                vix:=0;   vjx:=-1;
                viy:=1;   vjy:=0;
            end;
        Down:
            begin
                vX:=StoneBreite-1;
                vY:=StoneHoehe-1;
                vix:=-1; vjx:=0;
                viy:=0;  vjy:=-1;
            end;
        Left:
            begin
                vX:=StoneBreite div 2-StoneHoehe div 2;
                vY:=StoneHoehe div 2+StoneBreite div 2-1;
                vix:=0;   vjx:=1;
                viy:=-1; vjy:=0;
            end;
    end;
end;

procedure TestRotVect;
var Angel :tAngel;
begin
    window(10,10,40,20);
    writeln('testzsrtdfsdhjkjgkfdjshlk');

```



```

for Angel:=up to left do
begin
  SetRotationVectors(Angel);
  with RotVectors do
  begin
    writeln(vX);
    writeln(vY);
    writeln(vix,vjx,viy,vjy);
    writeln('-----');
  end;
end;
end;

{*****}
{*      Spielstein Zeichnen                      *}
{*****}
{*      PM: In: X/Y-Position  [Pixels]              *}
{*      Muster ID                                *}
{*      Farbe                                    *}
{*      Rotationsvektoren                        *}
{*      (C) 3.12.92                                *}
{*****}
procedure SetStone(X,Y :integer; Style,Color :Byte; Rotation :tRotVect);
var i,j      :byte;
    realX,realY :integer;
    stri      :string;
begin
  SetColor(Color);
  SetFillStyle(SolidFill,Color);
  For i:=0 to StoneBreite-1 do
  For j:=0 to StoneHoehe-1 do
  begin
    if Stones[Style,i+1,j+1] then
    with RotVectors do
    begin
      realX:=X+(vX+i*vix+j*vjx)*XStp;
      realY:=Y+(vY+i*viy+j*vjy)*YStp;
      bar(realX,realY,realX+XStp,realY+YStp);
    end;
  end;
end;
end;

```

```

procedure TestSetStone;
var Angel :tAngel;
    x,y,i :integer;
begin
    x:=0;
    For Angel:=up to left do
    begin
        x:=x+75;
        SetRotationVectors(Angel);
        y:=0;
        for i:=1 to AnzStones do
        begin
            Y:=y+30;
            SetStone(x,Y,i,red,RotVectors);
            PutPixel(X,Y,White);
            putPixel(X+StoneBreite*XStp div 2,Y+StoneHoehe*YStp div 2,Cyan);
        end;
    end;
end;

procedure TestSetStUndRotVect;
var x,y,
    Style,
    Style1,
    Color,
    x1,y1 :byte;
    Angel :tAngel;
    Zeichen :char;
begin
    ClearViewPort;
    Angel:=up; Color:=blue; Style:=1;
    SetRotationVectors(Angel);
    repeat
        x1:=x; y1:=y; Style1:=style;
        Zeichen := readkey;
        case Zeichen of
            '2': Y:=Y+1;
            '8': Y:=Y-1;
            '6': x:=x+1;
            '4': x:=x-1;
            '0': if Style=AnzStones then Style:=1

```

```

        else inc(Style);
        '5': if Angel=left then Angel:=up
              else Angel:=Succ(Angel);
        '1': if Color=MaxColors then Color:=0
              else inc(Color);
    end;
    SetStone(X1,y1,Style1,black,RotVectors);
    SetRotationVectors(Angel);
    SetStone(X,y,Style,Color,RotVectors);
until Zeichen='q';
end;

{*****}
{*      Spielstein L"schen                                     *}
{*****}
{*      PM: In: X/Y-Position  [Pixels]                         *}
{*      Muster ID                                               *}
{*      Farbe                                                    *}
{*      Rotationsvektoren                                       *}
{*                                                                *}
{*      (C) 3.12.92                                             *}
{*****}
procedure DelStone(X,Y :integer; Style,Color :Byte; Rotation :tRotVect);
var i,j      :byte;
    realX,
    realY   :integer;
    stri    :string;
begin
    SetColor(GrideColor);
    SetFillStyle(SolidFill,BGColor);
    For i:=0 to StoneBreite-1 do
    For j:=0 to StoneHoehe-1 do
    begin
        if Stones[Style,i+1,j+1] then
        with RotVectors do
        begin
            realX:=X+(vX+i*vix+j*vjx)*XStp;
            realY:=Y+(vY+i*viy+j*vjy)*YStp;
            bar(realX,realY,realX+XStp,realY+YStp);
            rectangle(realX,realY,realX+XStp,realY+YStp);
        end;
    end;
end;

```

end;

```
{*****}
{*      Titelbild darstellen      *}
{*****}
{*      *}
{*      (C) 30.11.92      *}
{*****}
```

procedure Titel;

var i :integer;

begin

{ **** Hintergrund Zeichnen **** }

ClearViewPort;

SetFillStyle(SolidFill,BGColor);

Bar(0,0,GetMaxX,GetMaxY);

{ **** Beschriftungen **** }

print('T E T R I S',100,20,GothicFont,HorizDir,7,StatusColor);

print('F1 : Play Tetris',350,150,TriplexFont,HorizDir,3,ChrColor);

print('F2 : Options',350,185,TriplexFont,HorizDir,3,ChrColor);

print('F3 : Help',350,220,TriplexFont,HorizDir,3,ChrColor);

print('ESC: Quit',350,255,TriplexFont,HorizDir,3,ChrColor);

print('(C) 1992 by Claudio Claudiofzky',50,
310,TriplexFont,HorizDir,1,red);

{ **** TopFive Zeigen **** }

ShowTopFive;

end;

```
{*****}
{*      Optionen verstellen      *}
{*****}
{*      *}
{*      (C) 10.12.92      *}
{*****}
```

procedure ShowOptions;

var sLevel,

sAInk :String[3];

begin

```

Str(Level,sLevel);
if AutoInk then sAink:='Yes'
else sAink:=' No';
SetColor(GrideColor);
SetLineStyle(DottedLn,0,3);
SetFillStyle(SolidFill,BGColor);
Bar(40,150,300,290);
print('Options',100,150,TriplexFont,HorizDir,1,ChrColor);
print('F1 = Level      :'+sLevel,50,180,TriplexFont,HorizDir,1,ChrColor);
print('F2 = Speed Up   :'+sAink,50,200,TriplexFont,HorizDir,1,ChrColor);
print('F3 = BackGround:',50,220,TriplexFont,HorizDir,1,ChrColor);
print('ESC= Quit Options',50,240,TriplexFont,HorizDir,1,ChrColor);
end;

procedure Options;
var Zeichen :char;
    LineStyle :word;
begin
    ShowOptions;
    repeat
        Zeichen:=' ';
        SetColor(StatusColor);
        SetLineStyle(SolidLn,0,3);
        Rectangle(40,150,300,290);
        LineStyle:=LineStyle*16;
        If LineStyle=0 then LineStyle:=15;
        SetColor(BGColor);
        SetLineStyle(UserBitLn,LineStyle,3);
        Rectangle(40,150,300,290);
        if keypressed then Zeichen:=readkey;
        if ord(Zeichen)=0 then Zeichen:=readkey;
        if ord(Zeichen) in [59,60,61] then
            begin
                case ord(Zeichen) of
                    59: if Level=5 then Level:=1
                        else inc(Level);
                    60: AutoInk:=not (AutoInk);
                    61: if BGColor=GetMaxColor-1 then BGColor:=AnzStones+1
                        else inc(BGColor);
                end;
            end;
        ShowOptions;
    end;
end;

```



```

    until ord(Zeichen)=27;
end;

{*****}
{*      Hilfe      *}
{*****}
{*                *}
{*      (C) 11.12.92      *}
{*****}
procedure ShowHlp;
begin
    SetColor(GrideColor);
    SetLineStyle(DottedLn,0,3);
    SetFillStyle(SolidFill,BGColor);
    Bar(40,150,300,290);
    print('Help',100,150,TriplexFont,HorizDir,1,ChrColor);
    print('Cursor: Left,Right,Drop',50,180,TriplexFont,HorizDir,1,ChrColor);
    print('Space : Rotate',50,200,TriplexFont,HorizDir,1,ChrColor);
    print('F1      : Pause...',50,220,TriplexFont,HorizDir,1,ChrColor);
    print('...any Key 2 continue',50,240,TriplexFont,HorizDir,1,ChrColor);
    print('ESC      : Quit Help',50,260,TriplexFont,HorizDir,1,ChrColor);
end;

procedure Help;
var Zeichen :char;
    LineStyle :word;
begin
    ShowHlp;
    Zeichen:=' ';
    repeat
        SetColor(StatusColor);
        SetLineStyle(SolidLn,0,3);
        Rectangle(40,150,300,290);
        LineStyle:=LineStyle*16;
        If LineStyle=0 then LineStyle:=15;
        SetColor(BGColor);
        SetLineStyle(UserBitLn,LineStyle,3);
        Rectangle(40,150,300,290);
        if keypressed then Zeichen:=readkey;
    until ord(Zeichen)=27;
    SetFillStyle(SolidFill,BGColor);
    Bar(40,150,300,290);

```

end;

```
{*****}
{*      Spielstein Zeichnen in Spielrahmen      *}
{*****}
{*      PM: In: X/Y-Position  [Spielfelder]      *}
{*              Muster ID                        *}
{*              Farbe                        *}
{*              Rotationsvektoren                *}
{*              *}
{*      (C) 3.12.92                        *}
{*****}
```

```
procedure SetStonesF(X,Y :byte; Style,Color :Byte; Rotation :tRotVect);
```

```
var PixX,PixY :integer;
```

```
begin
```

```
    PixX:=XField+x*XStp;
```

```
    PixY:=YField+y*YStp;
```

```
    SetStone(PixX,PixY,Style,Color,Rotation);
```

```
end;
```

```
{*****}
{*      Spielstein L'schen in Spielrahmen      *}
{*****}
{*      PM: In: X/Y-Position  [Spielfelder]      *}
{*              Muster ID                        *}
{*              Farbe                        *}
{*              Rotationsvektoren                *}
{*              *}
{*      (C) 3.12.92                        *}
{*****}
```

```
procedure DelStonesF(X,Y :byte; Style,Color :Byte; Rotation :tRotVect);
```

```
var PixX,PixY :integer;
```

```
begin
```

```
    PixX:=XField+x*XStp;
```

```
    PixY:=YField+y*YStp;
```

```
    DelStone(PixX,PixY,Style,Color,Rotation);
```

```
end;
```

```
{*****}
{*      Stein im Spielfeld speichern      *}
{*****}
{*      PM: In: X/Y-Position [Spielfelder]      *}
{*****}
```

```

{*          Rotationsvektoren          *}
{*          Style Index                 *}
{*                                     *}
{*      (C) 7.12.92                     *}
{*****}
Procedure InsStone(X,Y,Style :byte);
var i,j :integer;
begin
    for i:=0 to StoneBreite-1 do
        for j:=0 to StoneHoehe-1 do
            with RotVectors do
                if Stones[Style,i+1,j+1] then PlayField[X+vX+i*vix+j*Vjx,Y+vY+i*viy+j*vjy]:=Style;
end;

procedure TestInsStone;
var x,y :integer;
begin
    for x:=0 to XKloz-1 do
        for y:=0 to YKloz-1 do
            if PlayField[X,Y]<>0 then bar(XField+X*XStp,YField+Y*YStp,XField+(X+1)*XStp,YField+(Y+1)*YStp);
end;

{*****}
{*      Spielstein setzen m"glich?      *}
{*****}
{*      PM: In: X/Y-Position  [Spielfelder]  *}
{*          Rotationsvektoren                *}
{*          Style Index                     *}
{*      Out:Ja/Nein                         *}
{*                                     *}
{*      (C) 7.12.92                         *}
{*****}
function Possible(X,Y,Style :byte) :boolean;
var XPF,YPF,i,j :Integer;
begin
    possible:=true;
    for i:=0 to StoneBreite-1 do
        for j:=0 to StoneHoehe-1 do
            with RotVectors do
                begin
                    XPF:=X+vX+i*vix+j*vjx;
                    YPF:=Y+vY+i*viy+j*vjy;

```

```

    if Stones[Style,i+1,j+1] then
    begin
        if XPF<0 then Possible:=false;
        if XPF>XKloz-1 then Possible:=false;
        if YPF>YKloz-1 then Possible:=false;
        if PlayField[XPF,YPF]<>0 then Possible:=false;
    end;
end;

end;

function PossibleTST(X,Y,Style :byte) :String;
var XPF,YPF,i,j :Integer;
begin
    PossibleTST:='O.K.';
    for i:=0 to StoneBreite-1 do
    for j:=0 to StoneHoehe-1 do
    with RotVectors do
    begin
        XPF:=X+vX+i*vix+j*vjx;
        YPF:=Y+vY+i*viy+j*vjy;
        if (XPF<0) and Stones[Style,i+1,j+1] then PossibleTST:='linker Rand';
        if (XPF>XKloz-1) and Stones[Style,i+1,j+1] then PossibleTST:='recher Rand';
        if (YPF>YKloz-1) and Stones[Style,i+1,j+1] then PossibleTST:='unterer Rand';
        if (PlayField[XPF,YPF]<>0) and Stones[Style,i+1,j+1] then PossibleTST:='anderer Stein';
    end;
end;

end;

procedure TestPossible;
var x,y,Style,Style1,Color,x1,y1 :integer;
    Angel :tAngel;
    Zeichen :char;
    i,j :integer;
begin
    for i:=0 to XKloz do
    for j:=0 to YKloz do
    PlayField[i,j]:=0;
    ClearViewPort;
    ShowPlayField;
    Angel:=up; Color:=blue; Style:=1;
    SetRotationVectors(Angel);
    X:=1; Y:=1;
    SetStonesSF(X,y,Style,Color,RotVectors);

```

```

repeat
  x1:=x; y1:=y; Style1:=style;
  Zeichen := readkey;
  case Zeichen of
    '2': Y:=Y+1;
    '8': Y:=Y-1;
    '6': X:=X+1;
    '4': X:=X-1;
    '0': if Style=AnzStones then Style:=1
         else inc(Style);
    '5': if Angel=left then Angel:=up
         else Angel:=Succ(Angel);
    '.': if Color=MaxColors then Color:=0
         else inc(Color);
    '+': begin
           InsStone(X,Y,Style);
           x:=1; y:=1; x1:=x; y1:=y;
         end;
    '-': TestInsStone;
  end;
  DelStonesF(X1,y1,Style1,black,RotVectors);
  SetRotationVectors(Angel);
  SetStonesF(X,y,Style,Color,RotVectors);
  bar(1,1,150,20);
  print(PossibleTST(X,Y,Style),1,1,TriplexFont,HorizDir,1,Red);
until Zeichen='q';
end;

{*****}
{*      Volle Linien 1"schen      *}
{*****}
{*                                  *}
{*      (C) 7.12.92                *}
{*****}
procedure EraseLine(y :integer);
var realX,
    realY,
    x,cnt :integer;
begin
  for x:=0 to XKloz-1 do
    for cnt:=y downto 0 do
      begin

```



```

    if cnt>0 then PlayField[X,Cnt]:=PlayField[X,Cnt-1]
    else PlayField[X,Cnt]:=0;
    realX:=XField+X*Xstp;
    realY:=YField+Cnt*Ystp;
    if PlayField[X,Cnt]=0 then
    begin
        SetColor(GrdeColor);
        SetFillStyle(SolidFill,BGColor);
        bar(realX,realY,realX+XStp,realY+YStp);
        rectangle(realX,realY,realX+XStp,realY+YStp);
    end
    else
    begin
        SetFillStyle(SolidFill,PlayField[X,Cnt]);
        bar(realX,realY,realX+XStp,realY+YStp);
    end;
end;
end;

Function CheckLines :byte;
var x,y :integer;
    era :boolean;
begin
    CheckLines:=0;
    for y:=YKloz downto 1 do
    begin
        x:=XKloz-1;
        era:=true;
        repeat
            if PlayField[x,y]=0 then era:=false;
            dec(x);
        until (not(era)) or (x=-1);
        if era then
        begin
            EraseLine(y);
            inc(y);
            CheckLines:=CheckLines+1;
        end;
    end;
end;

{*****}

```

```

{ *      Status anzeigen      * }
{ ***** }
{ *      PM: In: Punkte      * }
{ *      Linienzahl          * }
{ *      Steinanzahl         * }
{ *      (C) 10.12.92        * }
{ ***** }
Procedure ShowStatus(Points,Lines,Stones,Takt,Lvl :LongInt);
var i,x,y :integer;
    Stri :String;
begin
    SetFillStyle(SolidFill,BGColor);
    for i:=0 to 2 do
        Bar(XChr,YField+(1+i*2)*YStpChr,XField-FieldRand-5,YField+(1+i*2)*YStpChr+YStpChr div 2);
        Str(Points:6,Stri);
        print(Stri,XChr,YField+YStpChr,TriplexFont,HorizDir,1,StatusColor);
        Str(Stones:6,Stri);
        print(Stri,XChr,YField+3*YStpChr,TriplexFont,HorizDir,1,StatusColor);
        Str(Lines:6,Stri);
        print(Stri,XChr,YField+5*YStpChr,TriplexFont,HorizDir,1,StatusColor);
        Str(Takt:6,Stri);
        bar(XField+(XKloz+5)*XStp,YField+3*YStpChr,GetMaxX,YField+4*YStpChr);
        print(Stri,XField+(XKloz+5)*XStp,YField+3*YStpChr,TriplexFont,HorizDir,1,StatusColor);
        Str(Lvl:6,Stri);
        bar(XField+(XKloz+5)*XStp,YField+5*YStpChr,GetMaxX,YField+6*YStpChr);
        print(Stri,XField+(XKloz+5)*XStp,YField+5*YStpChr,TriplexFont,HorizDir,1,StatusColor);
    end;

{ ***** }
{ *      Action Teil      * }
{ ***** }
{ *      (C) 30.11.92      * }
{ ***** }
function Play(Lvel :byte) :LongInt;
var Tkt,i,j,x,y,xl,yl,TktOrig :Integer;
    Score,Lines,Stones :LongInt;
    Stone,NextStone,FasterCnt,DiffLines :byte;
    KO,Next :boolean;
    Zeichen :char;
    Angel,Angell :tAngel;

```

```

begin
{ **** Spielfeld L"schen **** }

    for i:=0 to XKloz do
    for j:=0 to YKloz do
    PlayField[i,j]:=0;
    Next:=false; KO:=Next;
    TktOrig:=Tackt div Level;
    Score:=0;
    Lines:=0;
    Stones:=0;
    FasterCnt:=0;
    NextStone:=1;

{ **** Spiel Beginnt **** }

    ShowPlayField;
    repeat

{ **** N„chster stein ausw„hlen **** }

        Stone:=NextStone;
        NextStone:=Random(AnzStones)+1;
        if ShowNext then
        begin
            SetFillStyle(SolidFill,BGColor);
            bar(XField+(XKloz+4)*XStp,YField,XField+(Xkloz+8)*XStp,YField+StoneHoehe*2*YStp);
            SetStonesSF(XKloz+5,1,NextStone,NextStone,RotVectors);
        end;
        x:=XKloz div 2;
        y:=0;
        Tkt:=TktOrig;
        Angel:=up;
        SetRotationVectors(Angel);
        SetStonesSF(x,y,Stone,Stone,RotVectors);

{ **** Stein fallen lassen **** }

        repeat

{ **** Alte Koordinaten speichern **** }

```

```

        x1:=x; y1:=y; Angell:=Angel;

{ **** Tastatur abfragen **** }

        if keypressed then Zeichen := readkey;
        if ord(Zeichen)=0 then Zeichen := readkey;

{ **** Tastaturabfrage auswerten **** }

        case Ord(Zeichen) of
          77 : x:=x+1;
          75 : x:=x-1;
          32 : if Angel=left then Angel:=up
                else Angel:=Succ(Angel);
          80 : begin
                  Tkt:=0;
                  Score:=Score+500 div (y+1);
                end;
          59 : repeat
                  Until keypressed;
        end;
        Zeichen:='+';

{ **** Testen ob Bewegung m"glich **** }

        SetRotationVectors(Angel);
        if not(Possible(X,Y,Stone)) then

{ **** Wenn nicht, bleiben alte Koordinaten **** }

        begin
          x:=x1;
          Angel:=Angell;
        end;

{ **** Bewegung am Bildschirm ausf"hren **** }

        SetRotationVectors(Angell);
        DelStoneSF(x1,y1,Stone,Black,RotVectors);
        SetRotationVectors(Angel);
        SetStoneSF(x,y,Stone,Stone,RotVectors);

```

```

{ **** Testen ob Stein fallen m"glich **** }
    if not(Possible(X,Y+1,Stone)) then
{ **** Wenn nicht m"glich neuer Stein/Ende **** }
    begin
        Next:=true;
        if y<=2 then KO:=true;
    end
    else
{ **** Wenn m"glich, Stein fallen lassen **** }
    begin
        DelStoneSF(x,y,Stone,Black,RotVectors);
        inc(y);
        SetStoneSF(x,y,Stone,Stone,RotVectors);
    end;
{ **** Zeitverz"gerung **** }
    Delay(Tkt);
{ **** Wenn Stein fallen nicht m"glich => n"chster Stein ****}
    until Next;
    Next:=false;
{ **** Stein ins Spielfeld einf"gen **** }
    InsStone(X,Y,Stone);
{ **** Volle Linien l"schen **** }
    DiffLines:=CheckLines;
    Lines:=Lines+DiffLines;
    Score:=Score+Sqr(DiffLines)*50;
    Inc(Stones);
    Inc(FasterCnt);
    If FasterCnt=10 then

```



```

begin
    FasterCnt:=0;
    Dec(TktOrig);
end;
ShowStatus(Score,Lines,Stones,TktOrig,Tackt div TktOrig);
until KO;
print('Press ESC to Continue',10,0,GothicFont,HorizDir,5,ChrColor);
repeat
until ord(readkey)=27;
Play:=Score;
end;

{*****}
{*      Menu                                     *}
{*****}
{*
{*      (C) 7.12.92                             *}
{*****}
procedure Menu;
var Zeichen    :char;
    Score,i    :integer;
    LineStyle  :word;
    Angel      :tAngel;
begin
    Titel;
    LineStyle:=15;
    Angel:=Up;
    Zeichen:=' ';
    repeat
        SetColor(BGColor);
        SetLineStyle(SolidLn,0,3);
        Rectangle(40,20,600,100);
        Rectangle(320,150,600,290);
        Rectangle(40,150,300,290);
        Rectangle(40,300,600,340);
        LineStyle:=LineStyle*16;
        If LineStyle=0 then LineStyle:=15;
        SetColor(StatusColor);
        SetLineStyle(UserBitLn,LineStyle,3);
        Rectangle(40,20,600,100);
        Rectangle(320,150,600,290);
        Rectangle(40,150,300,290);

```

```

Rectangle(40,300,600,340);
if Angel=left then Angel:=up
else Angel:=Succ(Angel);
SetFillStyle(SolidFill,BGColor);
bar(0,105,GetMaxX,105+4*YStp);
SetRotationVectors(Angel);
for i:=1 to AnzStones do SetStone(I*GetMaxX div (AnzStones+1),105,i,i,RotVectors);
if keypressed then Zeichen:=readkey;
if ord(Zeichen)=0 then
begin
    Zeichen:=readkey;
    case ord(Zeichen) of
        59:begin
            AktuTopFive(Play(Level));
            Titel;
            Zeichen:=' ';
        end;
        60:begin
            Options;
            Titel;
            Zeichen:=' ';
        end;
        61:begin
            Help;
            ShowTopFive;
        end;
        62:Zeichen:=Chr(27);
    end;
end;
until Zeichen=Chr(27);
end;

{*****}
{*      Hauptprogramm      *}
{*****}
{*                               *}
{*      (C) 7.12.92        *}
{*****}

begin
    Init;
    InitStones(StoneBitMap);

```

```
Menu;  
CloseGraph;  
end.
```